St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai

NAAC A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Academic Year: 2025-2026 **Term**: Even (Jan. 2026 – Jun. 2026)
Class / Branch: SE – AIML **Semester**: IV
Course: Web Programming Lab. (AI4VS_LR4)
Date of Assignment: / /2026 **Date of Submission:** / /2026

Pre-Lab Exercises for Experiment-11

Part A: Understanding NoSQL and MongoDB

Task 1: Conceptual Preparation

Answer the following questions:

1. What is NoSQL?
2. How is NoSQL different from SQL databases?
3. What is MongoDB?
4. What is a document in MongoDB?
5. What is a collection?
6. What is BSON?
7. What are advantages of MongoDB?

Part B: MongoDB Practice (Without Express)

Task 2: Install and Explore MongoDB

1. Install MongoDB locally OR create a free account in **MongoDB Atlas**.
2. Create a new database named:
   - practiceDB
3. Create a collection:
   - students

Perform the following operations:

- Insert 3 student records manually.
- View inserted records.
- Delete one record.
- Update one record.

Submission:

Take screenshots of:

- Database creation
- Inserted documents
- Updated document

Part C: Mongoose Familiarization

Task 3: Setup Basic Backend with Mongoose

1. Create a new Node.js project.
2. Install:
    - express
    - mongoose
3. Connect to MongoDB database.
4. Confirm successful connection in terminal.

Questions:

- What happens if connection string is wrong?
- What is the purpose of mongoose?

Task 4: Create Schema and Model

1. Create a schema with fields:
    - name
    - email
    - age
2. Create a model using the schema.
3. Insert one record using the model.
4. Retrieve all records from collection.

Observation:

- Is the data stored in MongoDB?
- Does schema restrict data structure?

Part D: REST API Practice

Task 5: Create Basic REST APIs

Create the following routes:

1. POST → Add user
2. GET → Retrieve all users

Test using:

- Postman

Verify:

- Status codes
- JSON response format

Part E: React API Practice

Task 6: Fetch Data in React

1. Create a simple React app.
2. Create a component:
   - UserList
3. Fetch data from backend GET API.
4. Store data in state.
5. Display data using:
   - List OR Table

Observation:

- Does UI update after fetching data?
- What happens if backend is not running?

Part F: Data Persistence Practice

Task 7: Restart Server Test

1. Add user using POST request.
2. Restart backend server.
3. Fetch users again.

Question:

- Does data still exist? Why?

Task 8: Perform All CRUD Operations via API

Activities:

1. Extend the backend to include:
   - PUT → Update user by ID
   - DELETE → Delete user by ID
2. Test all four operations using Postman:
   - Create user (POST)
   - Retrieve users (GET)
   - Update user (PUT)
   - Delete user (DELETE)
3. Verify changes in MongoDB database.

Observations to Record:

- What status codes are returned for each operation?
- What happens if an invalid ID is used?
- Does the database reflect changes immediately?

Task 9: Implement Basic Validation in Mongoose

Activities:

1. Modify schema to:
    - Make name required
    - Make email required
    - Add minimum age validation
2. Try inserting:
    - Valid data
    - Invalid data (missing fields or incorrect age)
3. Observe error messages.

Questions to Answer:

- Does MongoDB allow invalid data without schema?
- How does Mongoose validation improve data quality?
- Where are validation errors displayed?

Task 10: Handle Errors Gracefully

Activities:

1. Add error handling in:
    - Database connection
    - POST route
    - GET route
2. Send incorrect requests from Postman:
    - Invalid ID
    - Missing fields
3. Return proper:
    - Status codes (400, 404, 500)
    - JSON error messages

Observations:

- What status code is returned for bad requests?
- What happens if MongoDB server is stopped?
- How does proper error handling improve API reliability?